

BUSN 41210: Financial Analytics

Lecture 4: Model Evaluation and Selection

Dacheng Xiu

Chicago Booth

September 14, 2026

© 2026 Dacheng Xiu, Chicago Booth Restricted Materials. All Rights Reserved.

Outline

1. F-test/Partial F-test
2. Principles of Model Selection
3. Information Criteria
 - ▶ AIC
 - ▶ BIC and Bayesian Model Selection
4. Screening
5. Stepwise Strategies
6. Training, Validation, and Test
7. In-sample (IS) vs Out-of-sample (OOS) R^2 s

Model Evaluation vs Model Selection

Aspect	Model Evaluation	Model Selection
Motivation	Assess the performance of a trained model on unseen data	Choose the best model or configuration from a set of candidate models
Goal	Measure generalization performance and compute evaluation metrics	Select the model that performs best on future data
Process	Use validation or test sets to evaluate model accuracy, error rates, etc.	Compare different models or hyperparameter settings based on evaluation metrics
Outcome	Understand how well the model might perform in real-world situations	Identify the best-performing model for deployment

F-test for Model Evaluation

Recall the Multiple Linear Regression model

$$Y = \beta_0 + \beta_1 X_1 \dots + \beta_d X_d + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2)$$

F-test concerns with a big picture question:

Is this regression worthwhile? Does it help us predict Y ?

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_d = 0$$

$$H_1 : \text{at least one } \beta_j \neq 0.$$

Assess if the overall model is statistically significant in explaining the data

F-test Statistic

Construct the F test statistic from regression ANOVA:

$$f = \frac{SSR/(p-1)}{SSE/(n-p)} = \frac{R^2/(p-1)}{(1-R^2)/(n-p)}$$

If f is big, then the regression is “working”:

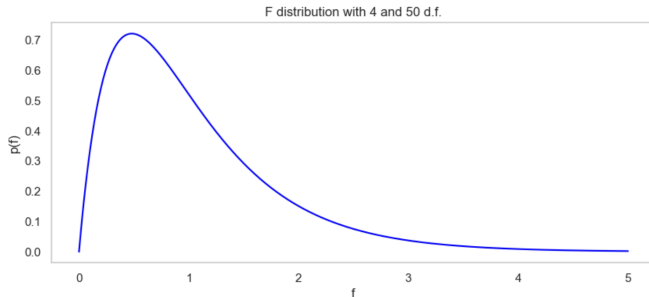
- ▶ Big SSR relative to SSE.
- ▶ R^2 close to one.

As usual, we need a statistical notion of “how big is big”. Under H_0 , f has $F_{p-1, n-p}$ distribution with $p-1$ numerator and $n-p$ denominator degrees of freedom.

- ▶ Generally, $f > 4$ is very significant (reject the null).

The F -Distribution

What kind of distribution is this?



It is a right skewed, positive valued family of distributions indexed by two parameters (the two df values).

The p -value for this test is $\varphi = \Pr(F_{p-1, n-p} > f)$.

Revisit the Market Equity Premium Example

```
> rest_full = F1.columns[0:-1].tolist()
> rest_full = '+' .join(rest_full)
> reg_full = smf.ols(formula='ret ~ {}'.format(rest_full), data = F1).fit()
> reg_full.summary()
```

OLS Regression Results						
Dep. Variable:		ret		R-squared:	0.419	
Model:		OLS		Adj. R-squared:	0.281	
Method:		Least Squares		F-statistic:	3.040	
Date:	Sun, 08 Sep 2024	Prob (F-statistic):		0.00142		
Time:	05:52:47	Log-Likelihood:		45.512		
No. Observations:	74		AIC:	-61.02		
Df Residuals:	59		BIC:	-26.46		
Df Model:		14				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.1315	0.456	2.481	0.016	0.219	2.044
dfy	9.4915	7.093	1.338	0.186	-4.701	23.684
infl	-2.0196	1.094	-1.846	0.070	-4.209	0.170
svar	1.5749	0.960	1.640	0.106	-0.347	3.496
d_e	-0.0248	0.069	-0.360	0.720	-0.163	0.113
lty	-0.8330	0.719	-1.158	0.251	-2.272	0.606
tms	-0.1880	1.067	-0.176	0.861	-2.324	1.948
tbl	-0.6451	0.792	-0.815	0.418	-2.229	0.939
dfr	0.3791	0.562	0.675	0.502	-0.745	1.503
d_p	0.1297	0.119	1.088	0.281	-0.109	0.368
d_y	-0.0127	0.150	-0.084	0.933	-0.314	0.288
ltr	0.1221	0.227	0.538	0.592	-0.332	0.576
e_p	0.1545	0.086	1.800	0.077	-0.017	0.326
b_m	-0.0574	0.229	-0.250	0.803	-0.516	0.401
ik	-2.8554	9.049	-0.316	0.753	-20.963	15.252
ntis	2.1022	1.319	1.594	0.116	-0.536	4.741
eqis	-0.9558	0.337	-2.833	0.006	-1.631	-0.281

F value of 3.040 is statistically significant (p -value = 0.00142).

Market Equity Premium

What about fitting a reduced model with only seven covariates?

```
> reg_reduced = smf.ols(formula='ret    infl + ik + eqis + d_p + ntis + e_p + svar', data=F1).fit()
> reg_reduced.summary()
```

OLS Regression Results			
Dep. Variable:	ret	R-squared:	0.387
Model:	OLS	Adj. R-squared:	0.322
Method:	Least Squares	F-statistic:	5.942
Date:	Sun, 08 Sep 2024	Prob (F-statistic):	2.22e-05
Time:	05:52:59	Log-Likelihood:	43.498
No. Observations:	74	AIC:	-71.00
Df Residuals:	66	BIC:	-52.56
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.2639	0.245	5.152	0.000	0.774	1.754
infl	-2.1795	0.779	-2.799	0.007	-3.734	-0.625
ik	-10.5915	6.538	-1.620	0.110	-23.644	2.461
eqis	-0.9050	0.288	-3.141	0.003	-1.480	-0.330
d_p	0.0843	0.082	1.033	0.305	-0.079	0.247
ntis	1.2801	1.068	1.199	0.235	-0.852	3.412
e_p	0.1411	0.084	1.671	0.099	-0.027	0.310
svar	2.2006	0.793	2.773	0.007	0.616	3.785

Market Equity Premium

The full model (16 covariates) has $R^2_{full} = 0.419$, while the second model (7 covariates) has $R^2_{base} = 0.387$.

Is this difference worth 9 extra covariates?

The R^2 will **always** increase as more variables are added

- ▶ If you have more b 's to tune, you can get a smaller SSE.
- ▶ Least squares fit “noise” in the data.
- ▶ This is known as **overfitting**.

More parameters will always result in a better fit to the sample data, but will not necessarily lead to better predictions.

Adjusted R^2

This is the reason some people like to look at adjusted R^2

$$R_a^2 = 1 - s^2/s_y^2$$

Since s^2/s_y^2 is a ratio of variance estimates, R_a^2 will not necessarily increase when new variables are added.

Unfortunately, the adjusted r-square is practically useless!

Problem: There is no theory for inference about R_a^2 , so we will not be able to tell “how big is big.”

Partial F -test

With the F -test example we were asking
“Is this regression worthwhile?”

Now, we're asking
“Is it useful to add these extra covariates to the regression?”

Partial F -test compares two **nested** models and decide if the additional predictors or complexity in the larger model are justified.

You **always** want to use the simplest model possible.

- ▶ Only add covariates if they are truly informative or compulsory.

Partial F -test

Consider the regression model

$$Y = \beta_0 + \beta_1 X_1 \dots + \beta_{d_{base}} X_{d_{base}} + \beta_{d_{base}+1} X_{d_{base}+1} \dots + \beta_{d_{full}} X_{d_{full}} + \varepsilon$$

Such that d_{base} is the number of covariates in the **base** (small) model and $d_{full} > d_{base}$ is the number in the **full** (larger) model.

The **Partial F -test** is concerned with the hypotheses

$$H_0 : \beta_{d_{base}+1} = \beta_{d_{base}+2} = \dots = \beta_{d_{full}} = 0$$

$$H_1 : \text{at least one } \beta_j \neq 0 \text{ for } j > d_{base}.$$

Partial F -test

It turns out that under the null H_0 (i.e. base model is true),

$$\begin{aligned} f &= \frac{(R_{full}^2 - R_{base}^2)/(d_{full} - d_{base})}{(1 - R_{full}^2)/(n - d_{full} - 1)} \\ &= \frac{(SSE_{full} - SSE_{base})/(p_{full} - p_{base})}{SSE_{full}/(n - p_{full})} \sim F_{p_{full} - p_{base}, n - p_{full}} \end{aligned}$$

That is, under the null hypothesis, the ratio of normalized $R_{full}^2 - R_{base}^2$ (increase in R^2) and $1 - R_{full}^2$ (full model $\frac{SSE}{SST}$) has F -distribution with $d_{full} - d_{base}$ and $n - d_{full} - 1$ df.

- ▶ Big f means that $R_{full}^2 - R_{base}^2$ is statistically significant.
- ▶ Big f means that at least one of the added X 's is useful.

Partial F -test in Python

Back to our Market Equity Premium data; we want to test

$H_0 : \beta_j = 0$ for all $j \in \{dfy, d_e, lty, tms, tbl, dfr, d_y, ltr, b_m\}$

$H_1 : \text{at least one } \beta_j \neq 0.$

The F -stat is $f = \frac{(R_{full}^2 - R_{base}^2)/(d_{full} - d_{base})}{(1 - R_{full}^2)/(n - d_{full} - 1)} = 0.36$

```
> n = reg_full.nobs
> R2_full = reg_full.rsquared
> R2_base = reg_reduced.rsquared
> d_full = reg_full.df_model
> d_base = reg_reduced.df_model

> F_statistic = ((R2_full - R2_base)/(d_full - d_base)) / ((1-R2_full)/(n - d_full - 1))
F_stat: 0.47
df1 = d_full - d_base
df2 = n - d_full - 1
p_value = 1 - f.cdf(F_statistic, df1, df2)
pvalue: 0.85
```

A p -value of 0.85 is not at all significant, so we stick with the null hypothesis and assume the base (7 covariates) model.

Market Equity Return: Partial F -test

As always, this is super easy to do in Python!

Just do ANOVA for the full model against the base model:

```
> anova_results = anova_lm(reg_reduced, reg_full)
> anova_results
```

df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
66.0	1.337192	0.0	NaN	NaN	NaN
59.0	1.266348	7.0	0.070844	0.471527	0.851311

F-test vs *t*-test

Suppose that you have d covariates, $X_1 \dots X_d$, and you are considering adding a new covariate X_{d+1} into your model.

The *t*-test is $H_0 : \beta_{d+1} = 0$ vs $H_1 : \beta_{d+1} \neq 0$, with test statistic $z = b_{d+1}/s_{b_{d+1}}$ and *p*-value $P(Z_{n-d-2} > |z|)$.

Partial *F*-test is $H_0 : \beta_{d+1} = 0$ vs $H_1 : \beta_{d+1} \neq 0$, with test stat $f = (SSE_{full} - SSE_{base})/MSE_{full}$ and *p*-value $P(F_{1,n-d-2} > f)$.

The test hypotheses are exactly the same!

F-test vs t-test

It turns out that the tests also lead to the same p -values.

Consider testing the two covariates (ik and ltr) regression against an even more simple model that includes only ik.

The t -test asks: is b_{ltr} far enough from zero to be significant?

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.8713	0.218	3.987	0.000	0.436	1.307
ik	-23.7512	6.057	-3.921	0.000	-35.829	-11.673
ltr	0.1158	0.172	0.675	0.502	-0.226	0.458

F-test asks: is the increase in R^2 significantly big?

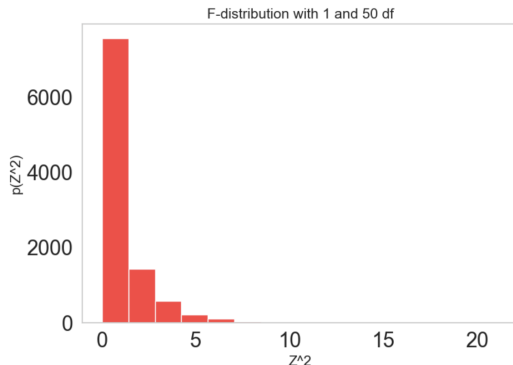
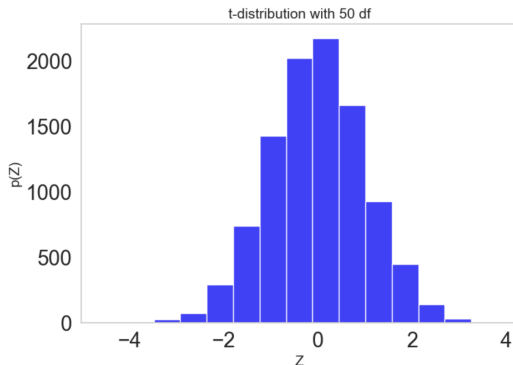
```
> anova_lm(reg_simple, reg_2cov) #ltr
```

df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
72.0	1.785140	0.0	NaN	NaN	NaN
71.0	1.773761	1.0	0.011379	0.455463	0.501944

F -test vs t -test

Ask the same question, F and t tests give the same answer!

In fact, $f = z^2$ and $P(Z_{n-d-2} > |z|) = P(F_{1,n-d-2} > z^2)$,
so the f stat for one extra variable is just a squared t stat.



F-test vs *t*-test

Why not always decide what to include by just looking at individual *t*-stats?

Multicollinearity: If the X 's are highly correlated with each other, then s_{b_j} 's will be very big (since you don't know which X_j to regress onto), and you will fail to reject $\beta_j = 0$ for all of the X_j 's even if they **do** have a strong effect on Y .

Does *F*-test significance imply at least one *t*-test significance? How about vice versa?

Market Equity Premium: F -test vs t -test

Does F -test significance imply at least one t -test significance?

```
> reg_temp = smf.ols(formula='ret ~ infl + lty + tbl + svar', data=F1).fit()
> reg_temp.summary()
```

OLS Regression Results			
Dep. Variable:	ret	R-squared:	0.177
Model:	OLS	Adj. R-squared:	0.130
Method:	Least Squares	F-statistic:	3.718
Date:	Sun, 08 Sep 2024	Prob (F-statistic):	0.00847
Time:	15:29:51	Log-Likelihood:	32.636
No. Observations:	74	AIC:	-55.27
Df Residuals:	69	BIC:	-43.75
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.0938	0.049	1.923	0.059	-0.004	0.191
infl	-0.1117	0.917	-0.122	0.903	-1.941	1.718
lty	0.2041	1.460	0.140	0.889	-2.709	3.117
tbl	-2.2283	1.521	-1.465	0.147	-5.262	0.806
svar	0.7355	0.739	0.995	0.323	-0.738	2.209

Market Equity Premium: F -test vs t -test

Does at least one t -test significance imply F -test significance?

```
> reg_temp = smf.ols(formula='ret ~ svar + d_y', data=F1).fit()  
> reg_temp.summary()
```

OLS Regression Results			
Dep. Variable:	ret	R-squared:	0.069
Model:	OLS	Adj. R-squared:	0.043
Method:	Least Squares	F-statistic:	2.624
Date:	Sun, 08 Sep 2024	Prob (F-statistic):	0.0795
Time:	06:08:31	Log-Likelihood:	28.053
No. Observations:	74	AIC:	-50.11
Df Residuals:	71	BIC:	-43.19
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.2508	0.157	1.600	0.114	-0.062	0.563
svar	1.6381	0.789	2.075	0.042	0.064	3.212
d_y	0.0759	0.047	1.623	0.109	-0.017	0.169

Variable Selection

Suppose we observe Y , an outcome of interest, and $(X_1, \dots, X_d)$, a set of potential explanatory variables or predictors.

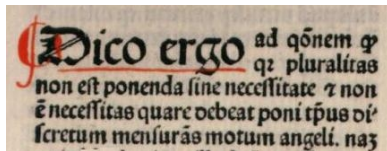
Assuming the model is always linear, so model selection simplifies to variable selection.

The variable selection problem:

To find the "best" subset of X 's for predicting Y

Particularly of interest when d is large and $X_1, \dots, X_d$ is thought to contain many redundant or irrelevant variables.

Guiding Principle for Variable Selection



Ockham's Razor: **Plurality ought not be posited without necessity.**

Brevity is the soul of wit.

Overly complicated models lead to bad forecasts.

Variable Selection

We need to find a trade-off between data fit and simplicity.

We already know one method: **partial F -test!**

- ▶ Do your regression with covariates you think should be in the model.
- ▶ Kick out the variables that don't seem significant.
- ▶ Use a partial F -test to see if the simple model is good enough.

This only works for a small number of variables, which you've chosen intelligently.

Multiple Testing

A big problem with using tests (t or F) for comparing models is the **false discovery rate** associated with **multiple testing**:

- ▶ If you do 20 tests of **true H_0** , with $\alpha = .05$, you expect to see 1 positive (i.e. you expect to reject at true null).

Suppose that you have 100 predictors, 10% of which are actually influential, and that you find all of those significant.

Since you reject H_0 for 5% of the useless 90% variables, $4.5/(4.5 + 10) \approx 1/3$ of your significant b_j 's will be false positives!

In modern big data problems, $<1\%$ of variables are influential.

Let Us Actually Try It

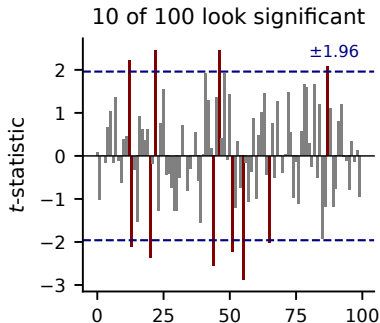
That was arithmetic. Here is the same thing done to us, on the market equity premium series we have been using all lecture.

The setup, and note how honest it is:

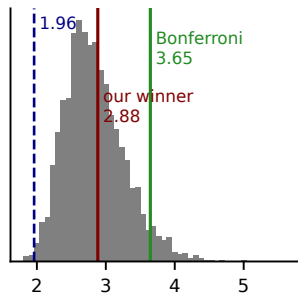
- ▶ Take the real series, $n = 74$ observations.
- ▶ Generate 100 predictors from a random number generator. Standard normal, independent of each other and of everything else.
- ▶ By construction, not one of them has any relationship to the market. There is no signal to find. We know this because we made them up.
- ▶ Regress the market on each one in turn and collect the 100 t -statistics.

How many of the 100 come back significant at the 5% level? And how big will the best t -statistic be? Say a number before I show you.

Ten of Them. And the Best Has $p = 0.005$.



the best of 100, under the null



- ▶ 10 of the 100 are significant at the 5% level — twice what the arithmetic predicted, which is itself just luck.
- ▶ The winner has $t = -2.88$, $p = 0.0052$. Written up on its own, with a story attached, that is a publishable-looking finding about a variable that *is definitionally noise*.
- ▶ And the right-hand panel says why it was inevitable. Over 4,000 repetitions of the whole exercise, the chance that *at least one* of 100 noise variables looks significant is 99.6%.

You will essentially always find something. The question was never whether you would.

And Adjusted R^2 Does Not Save You

Keep the best k of those hundred noise columns and fit them together:

best k noise columns	1	2	5	10	20
in-sample R^2	0.10	0.16	0.34	0.40	0.54
adjusted R^2	+0.09	+0.13	+0.29	+0.31	+0.37

An adjusted R^2 of 0.31 on a model containing no information whatsoever. The penalty for extra parameters is real, but it was never designed to protect you from having *searched*.

The fix is not a better statistic. It is a different critical value. Under the null, the largest of 100 t -statistics has a 95th percentile of **3.65** — not 1.96. Bonferroni gives 3.65 for 100 tests, which is the same number, and our winning t of 2.88 **does not clear it**.

Report the size of the search next to the result. And then, for the rest of this course, stop asking whether it fits and start asking whether it predicts.

Information Criteria for Model Selection

Instead of testing, there are **Information Criteria** metrics which attempt to quantify how well our model **would** have predicted the data (regardless of what you've estimated for the β_j 's).

The best of these is the **BIC: Bayes Information Criterion**, which is based on a “Bayesian” philosophy of statistics.

$$BIC = n \log(s^2) + p \log(n),$$

where $p = d + 1 =$ ‘degrees of freedom’ used in your model fit.

You want to choose the model that leads to **minimum** BIC.

AIC for Model Selection

Another popular metric is the Akaike Information Criterion:

$$AIC = n \log(s^2) + 2p$$

A general form for these criterion is $n \log(s^2) + kp$, where $k = 2$ for AIC and $k = \log(n)$ for BIC.

In Python, we can get the AIC and BIC as follows.

```
> reg_full = smf.ols(formula='ret ~ {}'.format(rest_full), data = F1).fit()
```

```
> full_AIC = reg_full.aic
```

```
> full_BIC = reg_full.bic
```

```
full model AIC: -61.02
```

```
full model BIC: -26.46
```

AIC prefers more complicated models than BIC, and it is not as easily interpretable.

BIC for Model Selection

One (very!) nice thing about the BIC is that you can interpret it in terms of **model probabilities**.

Given a list of possible models $\{M_1, M_2, \dots, M_R\}$, the probability that model i is correct is

$$P(M_i) \approx \frac{e^{-\frac{1}{2}BIC(M_i)}}{\sum_{r=1}^R e^{-\frac{1}{2}BIC(M_r)}} = \frac{e^{-\frac{1}{2}[BIC(M_i) - BIC_{min}]}}{\sum_{r=1}^R e^{-\frac{1}{2}[BIC(M_r) - BIC_{min}]}}$$

(Subtract $BIC_{min} = \min\{BIC(M_1) \dots BIC(M_R)\}$ for numerical stability.)

A/BIC for Model Selection

Consider our market equity premium regressions; we can compare the full model to the reduced model.

	full	reduced
AIC	-61.02	-71.00
BIC	-26.46	-52.56

BIC and AIC both agree with our F -testing selection (reduced).

BIC indicates that we are practically 100% sure the reduced model is better.

```
prob = np.exp(-0.5*(BIC-min(BIC))).round(4)
print(prob/prob.sum())
```

```
[0. 1.]
```

Thus BIC is an alternative to testing for comparing models.

- ▶ It is easy to calculate.
- ▶ You are able to evaluate model probabilities.
- ▶ There are no “multiple testing” type worries.
- ▶ It generally leads to more simple models than F -tests.

As with testing, you need to narrow down your options before comparing models.

What if there are too many possibilities?

Subset Selection

General idea:

- ▶ Search across all permutations of models
- ▶ Choose best model according to some criterion

Challenges:

- ▶ Set of models may be large
 - ▶ Best model of size k , for $k = 1, \dots, p$
 - ▶ Combinations are $\sum_{k \leq p} \binom{p}{k} = 2^p$

Example: Macro Forecasting

Forecast monthly growth rate of US industrial production using 119 macro variables

Dataset extends from February 1960 to December 2021 (743 monthly observations)

	growth	RPI	W875RX1	DPCERA3M086SBEA	CMRMTSPLx	RETAILx	IPFPNSS	IPFINAL	IPCONGD	IPDCONGD	...	PCEPI
0	-0.008939	0.003226	0.004661	0.002791	0.016963	0.026606	0.024095	0.029021	0.031235	0.103831	...	-0.001751
1	-0.009019	0.001147	0.000906	0.004361	0.014403	0.003696	-0.005685	-0.003435	-0.011457	-0.013855	...	0.001620
2	-0.007959	0.001877	0.000905	0.014089	-0.028040	-0.001102	-0.003427	-0.001150	0.001153	-0.019965	...	-0.000454
3	-0.001143	0.003465	0.003612	0.015302	0.009853	0.025903	0.002290	0.001150	0.006880	-0.001189	...	0.003036
4	-0.012653	0.002405	0.002427	-0.020351	-0.031518	-0.015047	0.005688	0.006862	0.005699	0.009455	...	-0.002587
5	-0.003476	0.000764	-0.000235	-0.000250	0.008553	-0.001364	-0.011412	-0.011468	-0.006841	-0.010641	...	-0.000581
6	-0.001163	0.001791	0.001914	0.002061	-0.006866	-0.010099	-0.005757	-0.005784	-0.009197	-0.038796	...	0.001351
7	-0.010521	-0.001597	-0.002540	-0.000936	-0.001265	0.003414	-0.002312	0.000000	0.002310	0.011064	...	-0.000132
8	-0.001176	0.001302	0.000274	0.004734	0.016090	-0.000935	-0.008135	-0.005817	-0.004621	-0.012298	...	-0.001093
9	-0.014216	0.002949	0.003125	0.004835	-0.019032	0.008766	0.005818	0.004659	0.010363	0.011078	...	0.000577

Summary for Full Regression

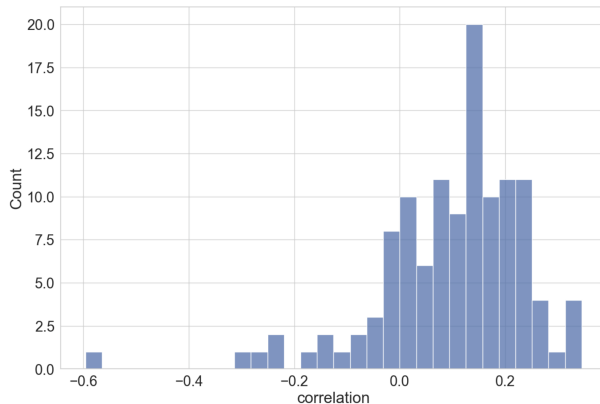
```
> rest_full = df.columns[1:].tolist()
> rest_full = '+' .join(rest_full)

> full_model = smf.ols(formula='growth ~ {}'.format(rest_full), data = df).fit()
> full_model.summary()
```

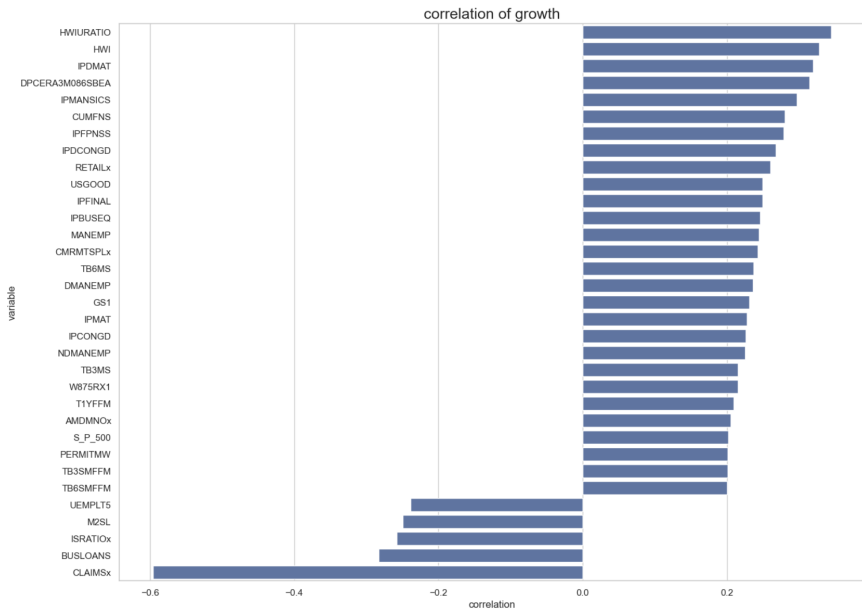
OLS Regression Results				coef	std err	t	P> t	[0.025	0.975]	
=====				Intercept	0.1423	0.055	2.601	0.010	0.035	0.250
Dep. Variable:	growth	R-squared:	0.624	RPI	-0.1838	0.067	-2.753	0.006	-0.315	-0.053
Model:	OLS	Adj. R-squared:	0.553	W875RX1	0.1266	0.059	2.164	0.031	0.012	0.242
Method:	Least Squares	F-statistic:	8.699	DPCERA3M086SBEA	0.1610	0.077	2.102	0.036	0.011	0.312
Date:	Sun, 08 Sep 2024	Prob (F-statistic):	3.55e-76	CMRMTSPLx	0.0296	0.045	0.661	0.509	-0.058	0.117
Time:	01:26:19	Log-Likelihood:	2750.5	RETAILx	-0.0678	0.036	-1.861	0.063	-0.139	0.004
No. Observations:	743	AIC:	-5261.	IPFPNSS	0.4865	0.282	1.723	0.085	-0.068	1.041
Df Residuals:	623	BIC:	-4708.	IPFINAL	-0.3209	0.294	-1.091	0.276	-0.898	0.257
Df Model:	119			IPCONGD	0.2672	0.342	0.782	0.435	-0.404	0.939
Covariance Type:	nonrobust			IPDCONGD	-0.0450	0.080	-0.562	0.574	-0.202	0.112
				IPNCONGD	-0.2912	0.236	-1.236	0.217	-0.754	0.171

Screening

The histogram of 119 correlations between each predictor and growth:



By only considering predictors with abs correlations greater than a certain threshold, we can reduce the total number of covariates.



Screening As A Selection Tool?

Maybe we grab models with covariates selected based on different correlation thresholds and compare?

Problems with screening:

It may keep redundant variables and omit variables that are important only when combined.

In post-screening regression, multicollinearity may occur and cause problems.

Bringing together variables that are useful individually does not mean that they will be useful collectively.

Forward Stepwise Regression

Forward stepwise procedures: start from a simple 'null' model, and incrementally update fit to allow slightly more complexity.

The `forward_selected()` function executes a common routine:

- ▶ Fit all univariate models. Choose that with lowest AIC and put that variable – say $x_{(1)}$ – in your model.
- ▶ Fit all bivariate models including $x_{(1)}$ ($y \sim \beta_{(1)}x_{(1)} + \beta_j x_j$), and add x_j from one with lowest AIC to your model.
- ▶ Repeat: min AIC by adding one variable to your model.

You stop when some model selection rule (AIC) is lower for the current model than for any of the models that add one variable.

Better than backwards methods

- ▶ The 'full' model can be expensive or tough to fit, while the null model is usually available in closed form.
- ▶ Jitter the data and the full model can change dramatically (because it overfits). The null model is always the same.

Stepwise approaches are 'myopic': they find the best solution at each step without considering the overall global path.

A related subtle (but massively important) issue is **stability**.

Stepwise procedure has high *sampling variability*: the selected variables change a lot from one dataset to another.

Out-of-Sample Prediction

How do we evaluate a prediction model? **Make predictions!**

Basic Idea: We want to use the model to predict outcomes for observations we have not seen before.

- ▶ Use the data to create a prediction problem.
- ▶ See how our candidate models perform.

We'll split the entire sample into training, validation, and test subsamples.

Training, Validation, and Test

1. Training Phase (model building)

- ▶ The model learns from the training set by adjusting its parameters (e.g., coefficients in a regression model).

2. Validation Phase (model selection)

- ▶ During training, different models or hyperparameters are evaluated using the validation set to determine the best model configuration.

3. Testing Phase (model evaluation)

- ▶ After model selection, the test set is used to evaluate the final model. This step provides a realistic assessment of how the model will perform on unseen data.
- ▶ The test set remains untouched until this stage to avoid biased estimates of model performance.

1: In-sample; 3: Out-of-sample;

2: gray zone. “In-” for model evaluation purpose, “Out-of-” for model selection.

IS R^2

It is common to use R^2 s as a measure of goodness of fit.

For **in-sample** R^2 , we have data $[X_1, Y_1] \dots [X_n, Y_n]$ and you use this data to fit $\hat{\beta}$. The SSE is then, say for linear regression,

$$\text{SSE}_{IS}(\hat{\beta}) = \sum_{i=1}^n (Y_i - X_i' \hat{\beta})^2.$$

The denominator is

$$\text{SSE}_{IS}(\hat{Y} = \bar{Y}) = \sum_{i=1}^n (Y_i - \bar{Y})^2.$$

The IS R^2 is given by:

$$R_{IS}^2 = 1 - \text{SSE}_{IS}(\hat{\beta}) / \text{SSE}_{IS}(\hat{Y} = \bar{Y}).$$

OOS R^2

For **out-of-sample** R^2 , observations $1 \dots n$ are still used to fit $\hat{\beta}$, but the SSE of the **numerator** is now calculated over *new* observations, say

$$\text{SSE}_{\text{OOS}}(\hat{\beta}) = \sum_{i \in \text{OOS}} (Y_i - X_i' \hat{\beta})^2$$

OOS R^2

The denominator choice for OOS R^2 has certain freedom.

- ▶ A common choice is based on $\bar{Y}$ from in sample data.

$$\text{SSE}_{OOS}(\hat{Y} = \bar{Y}) = \sum_{i \in OOS} (Y_i - \bar{Y})^2.$$

Why not out of sample $\bar{Y}$?

$$R_{OOS}^2 = 1 - \text{SSE}_{OOS}(\hat{\beta}) / \text{SSE}_{OOS}(\hat{Y} = \bar{Y}).$$

- ▶ Or one may use 0 or other estimators of β based on in sample data.

$$R_{OOS}^2 = 1 - \text{SSE}_{OOS}(\hat{\beta}) / \text{SSE}_{OOS}(0).$$

Ultimately, OOS R^2 is a comparison of SSEs between two predictors. It can be negative!

Market Equity Premium Example: OOS R^2

We'll split the **Market Equity Premium** data into a training set and a testing set, using an 80:20 ratio.

```
> N = len(F1)
> leaveout = range(4*N//5,N)
> yout = F1[['ret']].iloc[leaveout]
> yin = F1.drop(F1.index[leaveout])[['ret']]
> variable = '+'.join(F1.iloc[:, :-1].columns)
> ISreg = smf.ols(formula='ret ~ {}'.format(variable), data=F1.drop(F1.index[leaveout])).fit()
> IS_R2 = ISreg.rsquared
IS_R2: 0.47554

> y_pred = ISreg.predict(F1.iloc[leaveout])
> TSS = sse(yout.ret, yin.ret.mean())
> SSE = sse(yout.ret, y_pred)
> OOS_R2 = 1 - SSE/TSS
OOS_R2: -0.11806
```

Macro Forecasting Example: OOS R^2

Next, we'll split the [Macro Forecasting](#) data into a training set and a testing set, using an 80:20 ratio.

```
> N = len(df)
> leaveout = range(4*N//5,N)
> yout = df[['growth']].iloc[leaveout]
> yin = df.drop(df.index[leaveout])[['growth']]
> variable = '+'.join(df.iloc[:, :-1].columns)
> ISreg = smf.ols(formula='growth ~ {}'.format(variable), data=df.drop(df.index[leaveout])).fit()
> IS_R2 = ISreg.rsquared
IS_R2: 0.52849

> y_pred = ISreg.predict(df.iloc[leaveout])
> TSS = sse(yout.growth,yin.growth.mean())
> SSE = sse(yout.growth, y_pred)
> OOS_R2 = 1 - SSE/TSS
OOS_R2: -0.50138
```

Macro Forecasting Example: Screening and Forward Selection

The results appear to improve when using Correlation Screening or Forward Stepwise Selection.

	full	screening	forward
#variables	119.00	33.00	36.00
R2	0.62	0.47	0.58
OOS R2	-0.50	0.08	-0.23
AIC	-5260.94	-5177.60	-5336.11
BIC	-4707.65	-5020.84	-5165.51

However, both methods still present certain challenges...

Sensitivity to the Sample

Let's assume that 1 year of data is omitted.

```
> random_start = np.random.choice(df.index[:-11])  
> random_year = range(random_start, random_start + 12)  
> df_new = df.drop(index=random_year)
```

	full	screening	forward	screening w/ new data	forward w/ new data
#variables	119.00	33.00	36.00	30.00	41.00
R2	0.62	0.47	0.58	0.47	0.59
AIC	-5260.94	-5177.60	-5336.11	-5089.73	-5254.95
BIC	-4707.65	-5020.84	-5165.51	-4947.31	-5061.99

Our results indicate that both methods (Correlation Screening and Forward Selection) are sensitive to the sample data.